

HOSPITAL OPERATIONS & FINANCIAL PIPELINE

PHASE 1: DATA CLEANING & AUDIT REPORT

1. Core Business Problem Statement

The hospital network's operational database was facing a severe data trust crisis. Inconsistent system rules allowed corrupted entries—such as impossible patient ages scaling up to 150 and conflicting 1-to-10 customer satisfaction baselines—to quietly slip into leadership's weekly administrative tracking.

Furthermore, data gaps like missing procedure prices and unrecorded cities made it impossible for executives to identify top financial engines or accurately diagnose patient care bottlenecks.

The primary objective of this data cleaning pipeline is to eliminate these data-entry blind spots, secure data governance bounds, and establish an absolute "Source of Truth" for corporate decision-making.

2. Executive Summary

In healthcare, accurate data prevents costly business mistakes. This report details how we took a massive, unpolished dataset of hospital interactions and passed it through an automated cleaning pipeline. We fixed missing information and removed impossible records to build a trustworthy "Source of Truth" for leadership.

3. Data Overview & Quantities

- Total Starting Records: 101,000 individual entries (from 2022 to 2025).
- Attributes Tracked: 25 columns covering Patient Demographics (Age, Gender, Region), Hospital Operations (Dates, Departments, Booking Channels), and Financials (Costs, Revenue, Profit, Satisfaction Ratings).

ID	Date	Customer Name	Age	Gender	City	State	Region	Category	SubCategory	Metric1	Metric2	Metric3	Quantity	Cost	Price	Discount	Revenue	Profit	Rating	Channel	Status	Agent	Comments	
1	#####	C36763	Customer	61	Male	Delhi	Karnatak	West	B	Y	361	29243.1	18.8093	13	2463.17	28740.8	0.16153	7422.3	33872.8	1	Offline	Active	319	Average
2	#####	C44109	Customer	62	Female	Chennai	Delhi	South	B	X	743	18510.1	44.2171	12	9626.14	19617.3	0.00535	79631.3	28463.7	2	Offline	Inactive	246	Average
3	#####	C28588	Customer	57	Male	Hyderabad	Karnatak	North	D	Y	51	35944	15.3423	0	11721	7976.55	0.11761	60137.5	39098.8	1	Offline	Inactive	115	Average
4	#####	C20702	Customer	57	Male	Pune	Telangar	West	A	Z	640	1477.18	18.8106	6	1970.49	14256.6	0.20544	49514.9	43231.5	3	Offline	Inactive	43	Average
5	#####	C232	Customer	44	Female	Chennai	Tamil Na	South	B	Z	974	15563.2	24.0567	10	13038.6	1327.45	0.09118	17985.2	46241.7	1	Online	Active	221	Average
6	#####	C30633	Customer	29	Female	Mumbai	Delhi	West	C	Y	376	10907.4	25.1429	2	1275.33	6852.93	0.26252	58695.2	15195.1	5	Mobile	Pending	380	Average
7	#####	C45405	Customer	49	Female	Chennai	Tamil Na	South	C	X	572	29457.2	43.9848	13	17335.6	523.454	0.30627	18070	25065.9	3	Offline	Pending	403	Poor
8	#####	C7040	Customer	69	Female	Chennai	Telangar	East	C	Y	738	20413.4	82.4846	12	10290.5	2149.97	0.31395	87714.3	46709.7	4	Mobile	Inactive	257	Poor
9	#####	C12216	Customer	56	Female	Pune	Tamil Na	West	B	X	776	4620.26	9.66546	14	9392.81	16651.9	0.07452	48061.8	25095	1	Mobile	Inactive	101	Poor
10	#####	C26351	Customer	30	Male	Chennai	Delhi	South	B	Y	740	11493.1	88.7468	14	12595.1	N/A	0.17949	77299.7	10984.1	1	Mobile	Active	431	Average

4. Data Unification (Concatenation)

In hospitals, data is often split across different files by year. To combine them, we used a stacking function :-

→ **pd.concat** - This acts like stacking two decks of cards vertically, merging separate timelines into a single continuous ledger

→ We then reset the row numbers (**.reset_index**) to ensure clean formatting.

```
# 2. Re-indexing & Concatenation Simulation
df['Date'] = pd.to_datetime(df['Date'])
df_part1 = df[df['Date'].dt.year <= 2023]
df_part2 = df[df['Date'].dt.year > 2023]
df_cleaned = pd.concat([df_part1, df_part2], axis=0).reset_index(drop=True)
```

5. Step-by-Step Cleaning Functions

→ **Standardizing Dates (pd.to_datetime)**: Converted date text into proper "Timeline Objects" so the system can organize data chronologically by month and year.

```
df['Date'] = pd.to_datetime(df['Date'])
```

→ **Filling Blank Cities (.fillna)**: Repaired 3,035 records with blank cities by labeling them as 'Unknown'. This saves the financial data while flagging data-entry gaps for IT.

```
missing_city_before = df_cleaned['City'].isna().sum()  
df_cleaned['City'] = df_cleaned['City'].fillna('Unknown')
```

→ **Fixing Missing Prices (.groupby + .median)**: Corrected 3,040 missing prices by automatically grouping transactions by medical department and filling blanks with that specific department's middle (median) price.

```
missing_price_before = df_cleaned['Price'].isna().sum()  
df_cleaned['Price'] = df_cleaned.groupby('Category')['Price'].transform(lambda x: x.fillna(x.median()))
```

→ **Neutralizing Missing Discounts (.fillna(0))**: Found 1,513 blank discounts and filled them with 0, assuming no discount was given, which protects revenue calculations from math errors.

```
missing_discount_before = df_cleaned['Discount'].isna().sum()  
df_cleaned['Discount'] = df_cleaned['Discount'].fillna(0)
```

→ **Text Uniformity (.str.strip + .str.lower)**: Standardized messy text (e.g., converting " Good", "good", and "GOOD" into a single group: good) by removing accidental spaces and forcing lowercase.

```
df_cleaned['Comments'] = df_cleaned['Comments'].str.strip().str.lower()
```

6. Anomaly & Outlier Removal

We applied strict boundaries to purge impossible system glitch data:

- Ages: Removed 2,014 records where patient ages were listed between 101 and 150.
- Ratings: Hospital feedback uses a 1-to-5 star scale. We dropped 2,015 records where scores mistakenly scaled up to 10.

```
df_cleaned = df_cleaned[(df_cleaned['Age'] <= 100) & (df_cleaned['Rating'] <= 5)]
```

7. Final Clean Metrics

- Starting Volume: 101,000 rows
- Anomalies Dropped: 4,029 rows
- Final Verified Dataset: 96,971 perfectly clean, fully trusted records.

```
===== DATA CLEANING PIPELINE STARTED =====
```

```
===== PIPELINE SUCCESSFUL =====
```

```
Total Records Processed: 96032
```

```
Missing Cities Resolved: 3035
```

```
Missing Prices Imputed: 3040
```

```
Missing Discounts Handled: 1513
```

```
Duplicate Rows Removed: 972
```